

Class Activity – 19/08/2026

Name: Jana Malakondaiah

Reg No: 192424223

1. A retail chain's finance team wants to understand how daily sales have moved over the past two years before setting next year's budget. Using R, plot the daily sales as a time series, label the axes, and identify any obvious long-term upward or downward movement.

Code:

```
set.seed(10)
```

```
sales <- 1000 + cumsum(rnorm(730, mean = 2, sd = 30))
```

```
sales_ts <- ts(sales, frequency = 365)
```

```
plot(sales_ts,
```

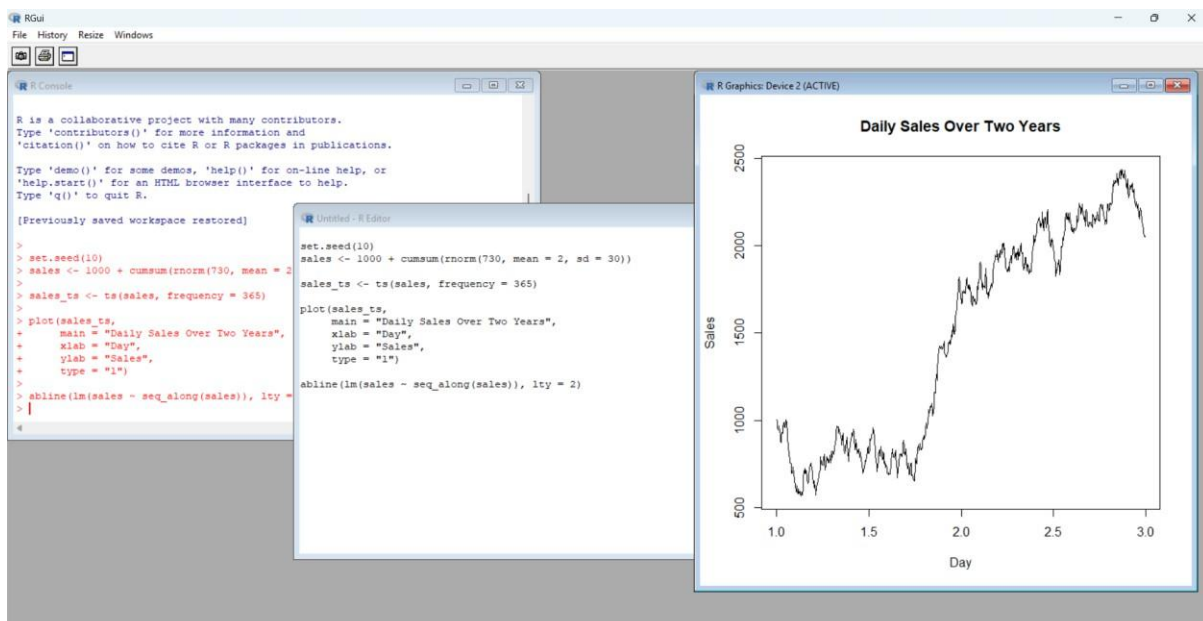
```
  main = "Daily Sales Over Two Years",
```

```
  xlab = "Day",
```

```
  ylab = "Sales",
```

```
  type = "l")
```

```
abline(lm(sales ~ seq_along(sales))), lty = 2)
```



2. An IoT company monitors hourly server temperature at a data center to catch overheating risks early. Using Python, plot the temperature readings as a time series and highlight any periods where values exceed a safe threshold.

Code:

```
import numpy as np

import matplotlib.pyplot as plt

np.random.seed(10)

temperature = 65 + np.random.normal(0, 3, 100)

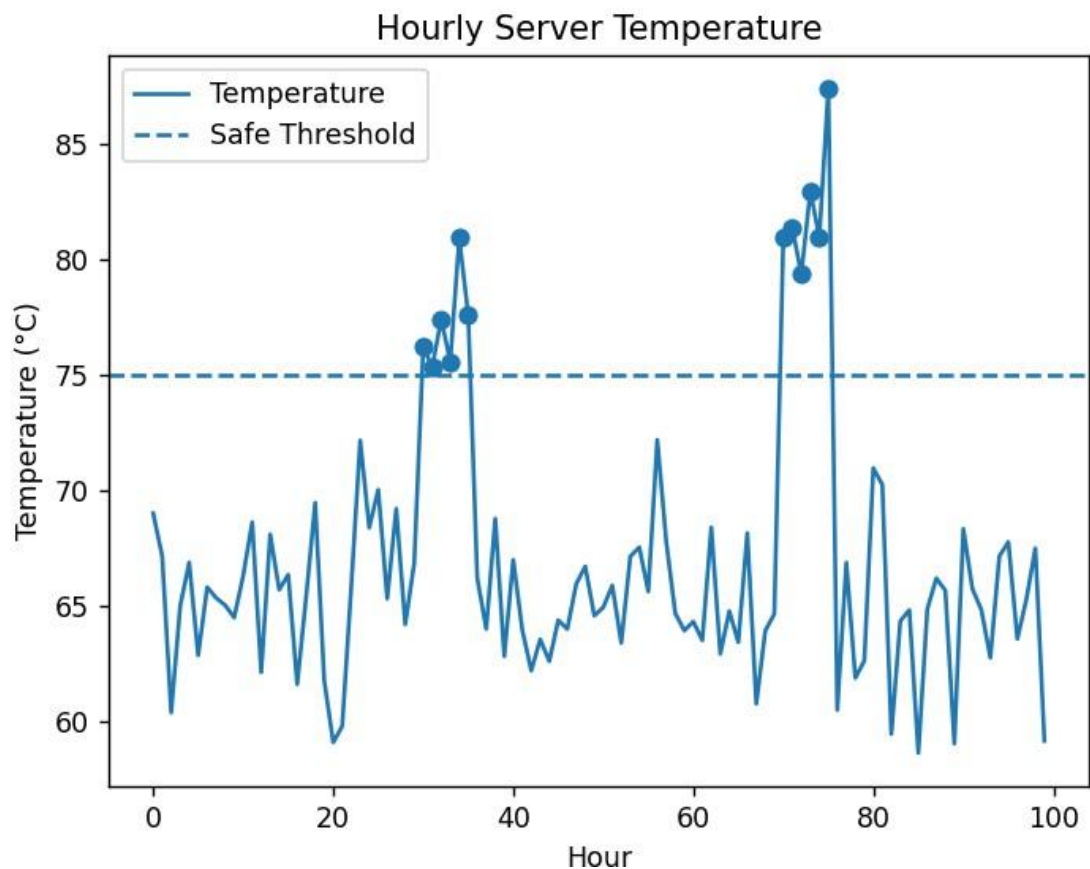
temperature[30:36] += 12
temperature[70:76] += 15

threshold = 75

plt.plot(temperature, label="Temperature")
plt.axhline(threshold, linestyle="--", label="Safe Threshold")

plt.scatter(np.where(temperature > threshold),
            temperature[temperature > threshold])

plt.xlabel("Hour")
plt.ylabel("Temperature (°C)")
plt.title("Hourly Server Temperature")
plt.legend()
plt.show()
```



3. A hedge fund analyst is comparing the daily closing prices of three competing tech stocks to decide which one shows steadier growth. Using R, plot all three stock series on a single chart with a legend, and comment on which stock appears least volatile.

Code:

```
set.seed(10)
```

```
stock1 <- 100 + cumsum(rnorm(100, 0.5, 2))
```

```
stock2 <- 100 + cumsum(rnorm(100, 0.5, 1))
```

```
stock3 <- 100 + cumsum(rnorm(100, 0.5, 3))
```

```
matplot(1:100,
```

```
  cbind(stock1, stock2, stock3),
```

```
  type = "l",
```

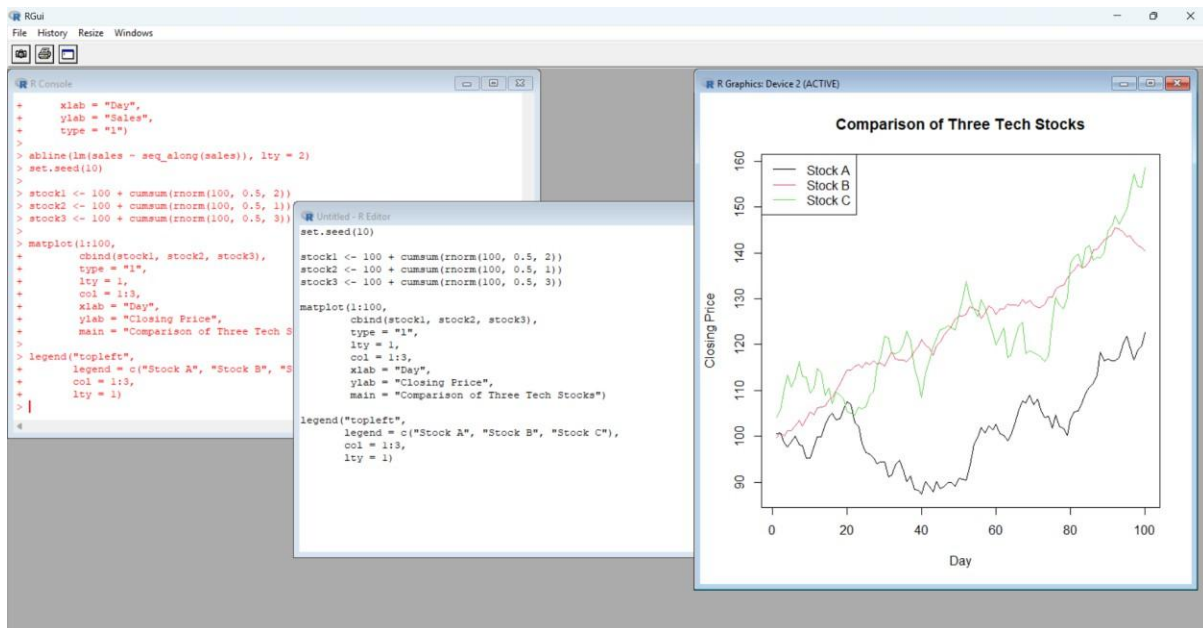
```
  lty = 1,
```

```
  col = 1:3,
```

```
  xlab = "Day",
```

```
ylab = "Closing Price",
main = "Comparison of Three Tech Stocks")
```

```
legend("topleft",
      legend = c("Stock A", "Stock B", "Stock C"),
      col = 1:3,
      lty = 1)
```



4. A city's energy board wants to compare monthly electricity consumption across four different neighborhoods to plan targeted conservation campaigns. Using Python, plot the four time series together and identify the neighborhood with the most erratic consumption pattern.

Code:

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
np.random.seed(10)
```

```
months = np.arange(1, 25)
```

```
north = 400 + np.random.normal(0, 20, 24)
```

```
south = 450 + np.random.normal(0, 15, 24)
```

```
east = 350 + np.random.normal(0, 50, 24)
```

```
west = 420 + np.random.normal(0, 25, 24)
```

```
plt.plot(months, north, label="North")
```

```
plt.plot(months, south, label="South")
```

```
plt.plot(months, east, label="East")
```

```
plt.plot(months, west, label="West")
```

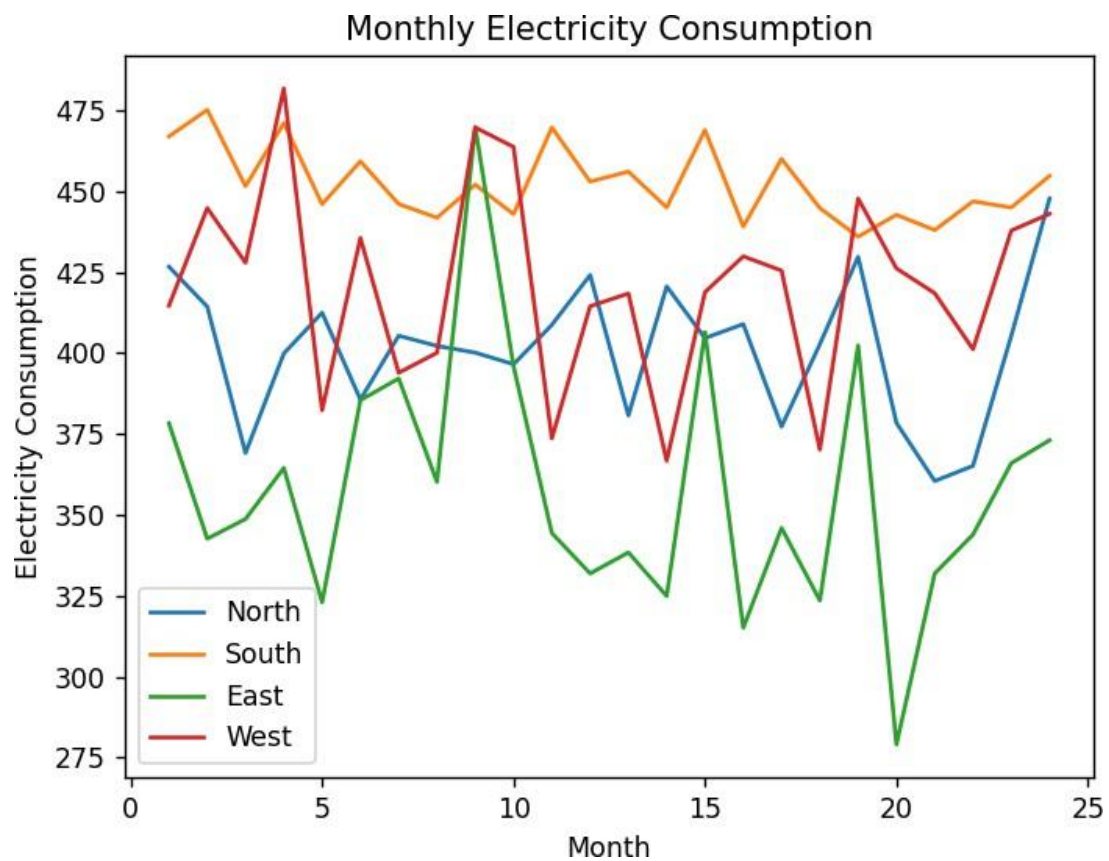
```
plt.xlabel("Month")
```

```
plt.ylabel("Electricity Consumption")
```

```
plt.title("Monthly Electricity Consumption")
```

```
plt.legend()
```

```
plt.show()
```



5. A pharmaceutical company is testing a new drug and needs to determine the minimum effective dose before advancing to clinical trials. Using R, plot a dose–response curve from the trial data (dose vs. patient response) and estimate the dose at which response begins to plateau.

Code:

```
dose <- c(0, 10, 20, 30, 40, 50, 60, 80, 100)
```

```
response <- c(5, 20, 38, 55, 68, 78, 84, 88, 90)
```

```
plot(dose, response,
```

```
  pch = 19,
```

```
  xlab = "Drug Dose (mg)",
```

```
  ylab = "Patient Response",
```

```
  main = "Dose-Response Curve")
```

```
model <- nls(response ~ a * (1 - exp(-b * dose)),
```

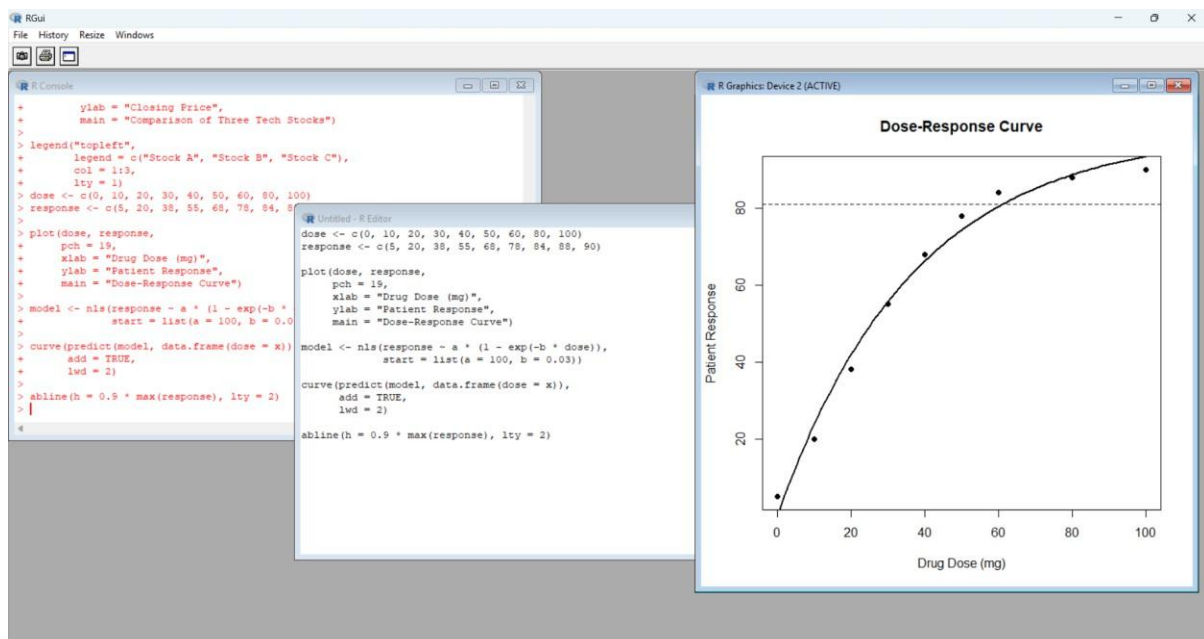
```
  start = list(a = 100, b = 0.03))
```

```
curve(predict(model, data.frame(dose = x)),
```

```
  add = TRUE,
```

```
  lwd = 2)
```

```
abline(h = 0.9 * max(response), lty = 2)
```



6. An agricultural research lab is studying how increasing fertilizer concentration affects crop yield, aiming to avoid wasteful over-application. Using Python, fit and plot a dose–response curve of fertilizer amount vs. yield, and advise the optimal dosage range.

Code:

```
import numpy as np

import matplotlib.pyplot as plt

from scipy.optimize import curve_fit

fertilizer = np.array([0, 20, 40, 60, 80, 100, 120])
yield_data = np.array([20, 35, 48, 58, 64, 67, 68])

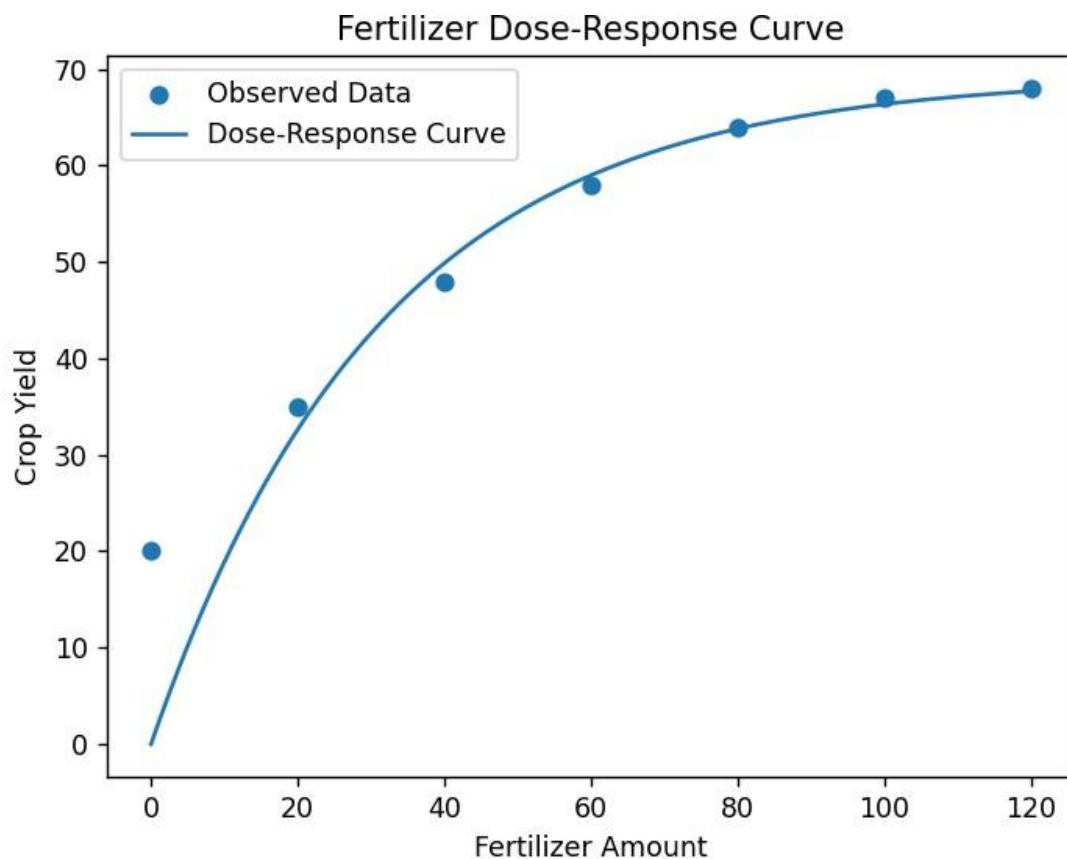
def response(x, a, b):
    return a * (1 - np.exp(-b * x))

params, _ = curve_fit(response, fertilizer, yield_data,
                      p0=[70, 0.03])

x = np.linspace(0, 120, 200)
y = response(x, *params)

plt.scatter(fertilizer, yield_data, label="Observed Data")
plt.plot(x, y, label="Dose-Response Curve")

plt.xlabel("Fertilizer Amount")
plt.ylabel("Crop Yield")
plt.title("Fertilizer Dose-Response Curve")
plt.legend()
plt.show()
```



7. An airline's revenue team suspects that ticket sales follow a strong seasonal pattern tied to holidays, but a slow underlying decline is being masked by this seasonality. Using R, apply STL decomposition to monthly ticket sales and separate the trend, seasonal, and residual components.

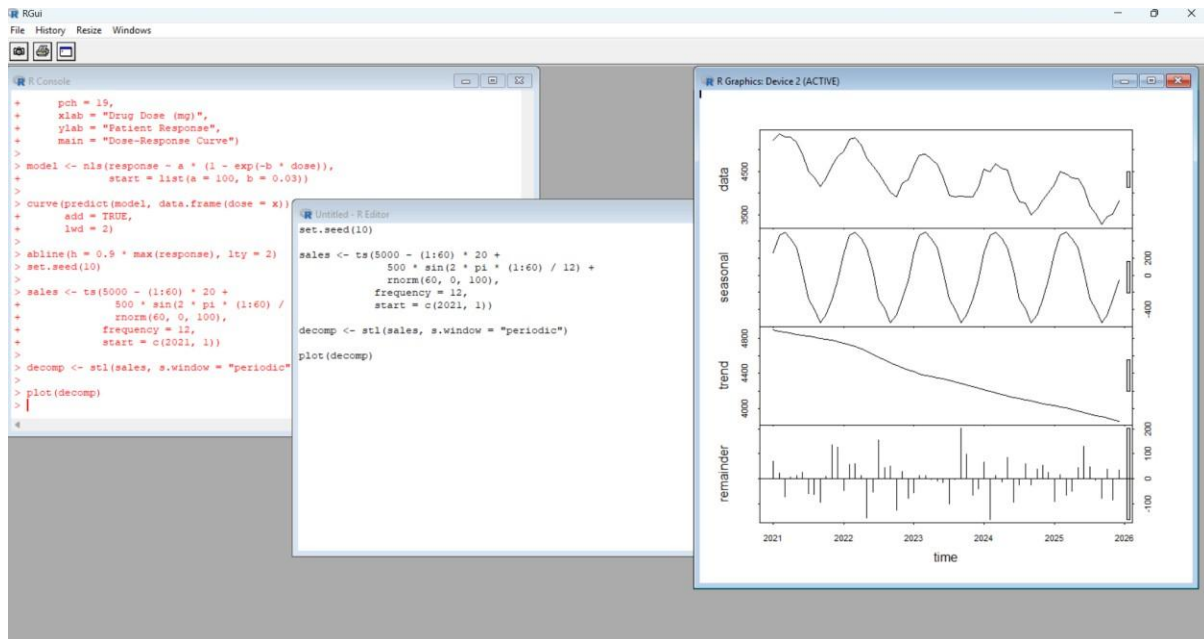
Code:

```
set.seed(10)
```

```
sales <- ts(5000 - (1:60) * 20 +  
           500 * sin(2 * pi * (1:60) / 12) +  
           rnorm(60, 0, 100),  
           frequency = 12,  
           start = c(2021, 1))
```

```
decomp <- stl(sales, s.window = "periodic")
```

```
plot(decomp)
```

8. A streaming platform wants to know whether recent dips in weekly active users reflect areal decline or just normal seasonal viewing habits. Using Python, perform STL decomposition on the weekly user-activity data and interpret the trend component to advise the product team.

Code:

```

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from statsmodels.tsa.seasonal import STL

np.random.seed(10)

users = 50000 - np.arange(104) * 50
users = users + 3000 * np.sin(2 * np.pi * np.arange(104) / 52)
users = users + np.random.normal(0, 800, 104)

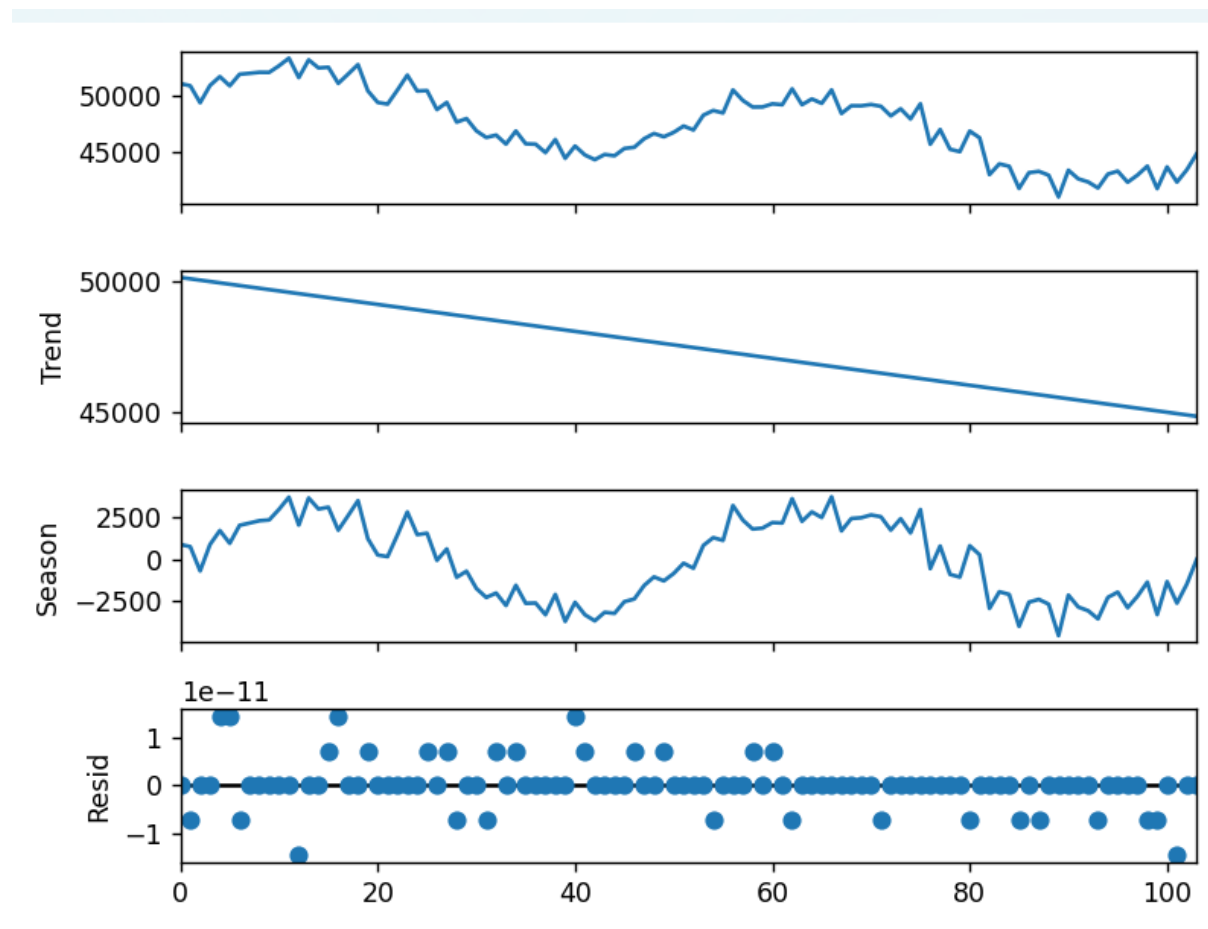
series = pd.Series(users)

result = STL(series, period=52).fit()

result.plot()

```

```
plt.show()
```



9. A stock trading firm wants to reduce short-term noise in a volatile stock's price data to spot the underlying momentum before making buy/sell calls. Using R, compute and plot a 7-day and 30-day moving average over the raw price series, and explain what the crossover between them suggests.

Code:

```
set.seed(10)
```

```
price <- 100 + cumsum(rnorm(100, 0.3, 3))
```

```
ma7 <- filter(price, rep(1/7, 7))
```

```
ma30 <- filter(price, rep(1/30, 30))
```

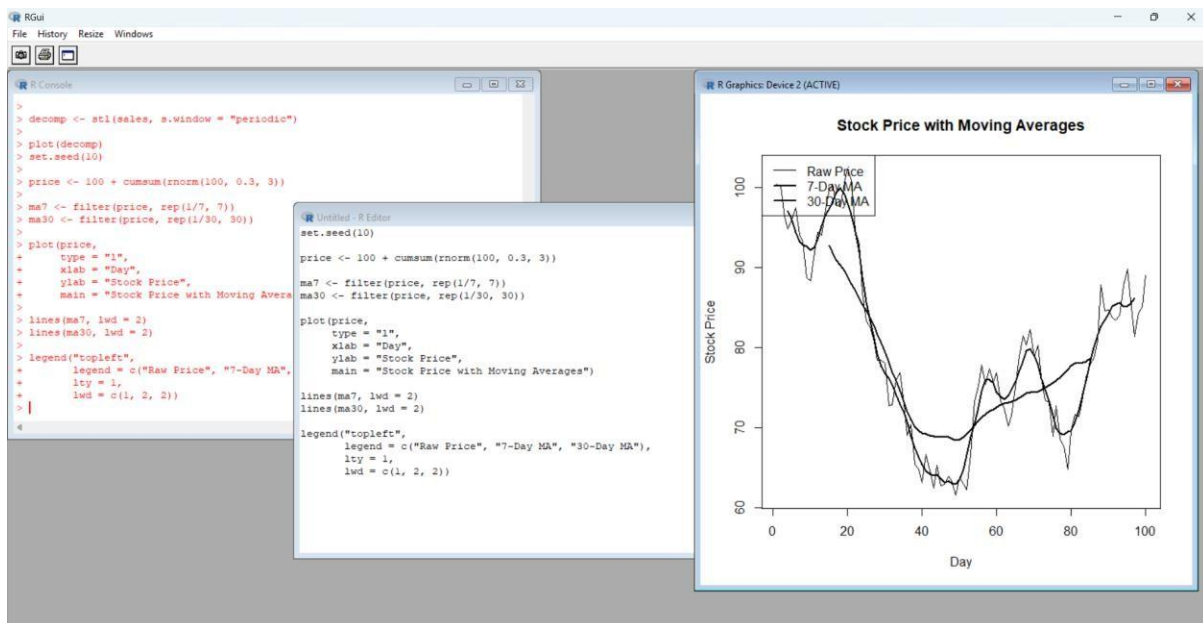
```
plot(price,  
      type = "l",  
      xlab = "Day",
```

```
ylab = "Stock Price",
main = "Stock Price with Moving Averages")
```

```
lines(ma7, lwd = 2)
```

```
lines(ma30, lwd = 2)
```

```
legend("topleft",
      legend = c("Raw Price", "7-Day MA", "30-Day MA"),
      lty = 1,
      lwd = c(1, 2, 2))
```



10. An economist is analyzing a country's quarterly GDP data but needs to remove the long- term growth trend to study business-cycle fluctuations in isolation. Using Python, detrend the GDP series (e.g., by differencing or regression-based detrending), plot the detrended series, and interpret the remaining fluctuations.

Code:

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.signal import detrend
```

```
gdp = np.array([
    100, 102, 105, 107,
```

110, 112, 115, 118,

120, 119, 123, 126,

128, 127, 131, 134,

136, 138, 137, 141

])

```
detrended = detrend(gdp)
```

```
plt.plot(detrended)
```

```
plt.axhline(0, linestyle="--")
```

```
plt.xlabel("Quarter")
```

```
plt.ylabel("Detrended GDP")
```

```
plt.title("Detrended Quarterly GDP")
```

```
plt.show()
```

